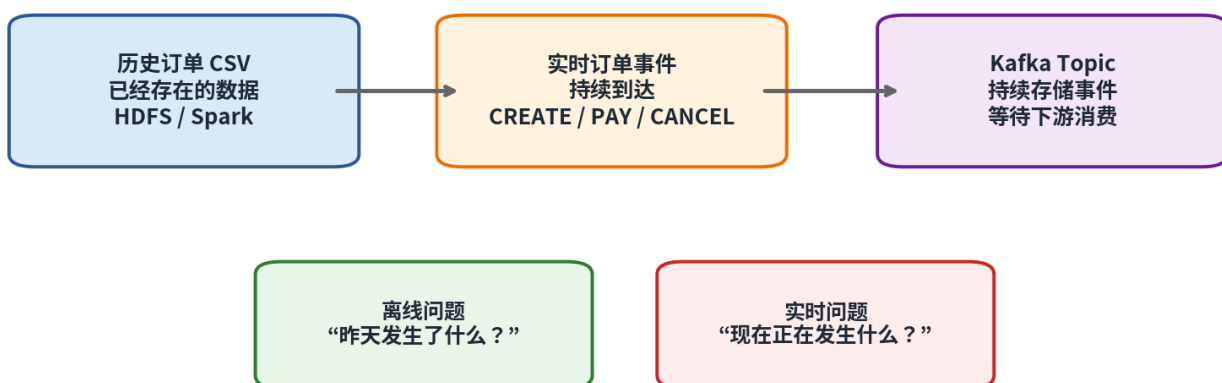


大数据综合项目实战

第6讲：Kafka 实时事件链 - Topic、Partition、Offset、Consumer Group 与 Lag

先把 Kafka 单独跑通，再让 Flink 接入

综合项目第6讲：从“历史文件”进入“实时事件流”



这一讲先只证明 Kafka 这一层可靠：能写、能存、能读、能看 Offset/Lag。暂时不接 Flink。

本讲最终验收

能独立证明：Kafka Broker 正常；orders_raw 有 3 个 Partition；8 条基础订单事件可被生产并完整读回；同一个 order_id 使用相同 Key；Consumer Group 的 CURRENT-OFFSET / LOG-END-OFFSET / LAG 能够解释。

本讲不启动 HDFS/YARN

Kafka 实验暂时不依赖 HDFS、YARN、Spark。不要为了“看起来完整”启动无关服务；只启动本讲真正需要的 Kafka。

1. 为什么综合项目现在要进入 Kafka ？

前 5 讲处理的是“已经存在”的历史文件：CSV 放进 HDFS，再由 MapReduce/Spark 读取。真实电商系统每天还有源源不断的新订单、支付、取消等事件。Kafka 解决的不是“把历史文件算一遍”，而是让这些不断发生的事件有一个可持续写入、可持久保存、可被多个下游独立读取的位置。

Producer → Broker/Topic → Consumer



Kafka 的关键不是“把消息立刻推给某个消费者”，而是先把事件追加到分区日志中。

因此消息可以被不同 Consumer Group 以自己的进度反复读取。

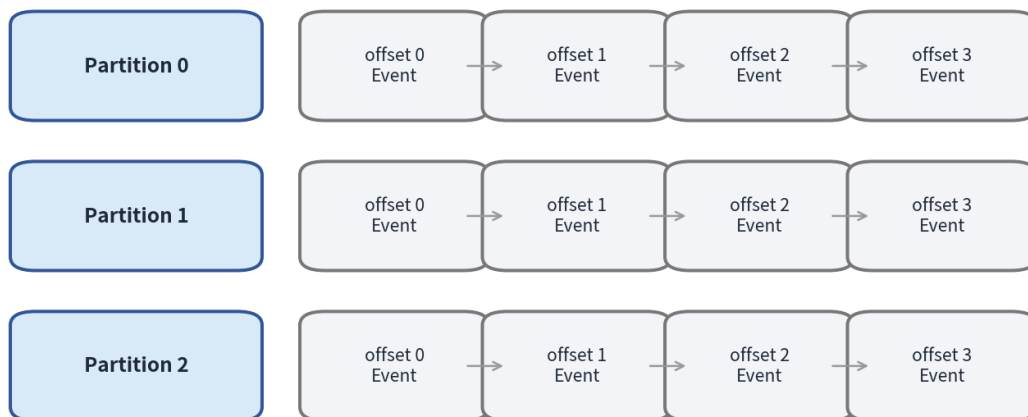
一句话理解 Kafka

Kafka 可以先理解成“按 Topic 分类、按 Partition 追加写入的持久化事件日志”。Producer 负责写；Broker 负责保存；Consumer 按自己的进度读。

2. 先认识本讲 6 个必须会说清楚的词

概念	通俗解释	本项目例子
Broker	Kafka 服务节点，接收并保存事件	localhost:9092 上的单机实验 Broker
Topic	一类事件的逻辑名字	orders_raw
Partition	Topic 内部的并行日志分片	orders_raw 的 3 个 Partition
Offset	一条消息在某个 Partition 内的位置	Partition 1 的 offset 7
Message Key	决定同类事件如何分区的重要字段	order_id
Consumer Group	一组消费者共享一套消费进度	capstone-l6

Topic、Partition、Offset：Kafka 最核心的三个名词



Topic 是逻辑容器；Partition 是可并行的有序日志；Offset 是消息在某个 Partition 内的位置。

最容易犯的错误

Offset 不是 Topic 的“全局第几条消息”。Offset 只在某个 Partition 内有意义；不同 Partition 都可以同时存在 offset=0。

3. 本讲继续使用第 1 讲的 8 条实时事件

Starter Pack 里的 data/order_events.jsonl 已经定义了 8 条事件。JSONL（JSON Lines）表示“一行一个 JSON 对象”，非常适合逐条发送到 Kafka。

```
{ "event_id": "E001", "order_id": "02001", "user_id": "U001", "city": "Beijing", "amount": 258.0, "event_type": "CREATE", "event_time": "2026-09-29 10:00:05" }
{ "event_id": "E002", "order_id": "02002", "user_id": "U002", "city": "Shanghai", "amount": 499.0, "event_type": "CREATE", "event_time": "2026-09-29 10:00:12" }
{ "event_id": "E003", "order_id": "02001", "user_id": "U001", "city": "Beijing", "amount": 258.0, "event_type": "PAY", "event_time": "2026-09-29 10:01:10" }
{ "event_id": "E004", "order_id": "02003", "user_id": "U004", "city": "Beijing", "amount": 118.0, "event_type": "CREATE", "event_time": "2026-09-29 10:00:40" }
{ "event_id": "E005", "order_id": "02002", "user_id": "U002", "city": "Shanghai", "amount": 499.0, "event_type": "PAY", "event_time": "2026-09-29 10:01:30" }
{ "event_id": "E006", "order_id": "02004", "user_id": "U003", "city": "Guangzhou", "amount": 5999.0, "event_type": "CREATE", "event_time": "2026-09-29 10:02:00" }
{ "event_id": "E007", "order_id": "02004", "user_id": "U003", "city": "Guangzhou", "amount": 5999.0, "event_type": "CANCEL", "event_time": "2026-09-29 10:03:00" }
{ "event_id": "E008", "order_id": "02003", "user_id": "U004", "city": "Beijing", "amount": 118.0, "event_type": "PAY", "event_time": "2026-09-29 10:02:20" }
```

业务顺序不等于文件顺序

注意 E004 的 event_time=10:00:40，却排在 E003（10:01:10）后面。这是故意保留的乱序特征。Kafka 本讲只负责按写入顺序保存；“事件时间乱序如何计算”留给后面的 Flink。

4. 保姆式起手式：启动 VM → VS Code Remote SSH → 找到 Kafka

步骤 1：启动 Ubuntu 22.04 虚拟机。步骤 2：Windows 打开 VS Code。步骤 3：使用 Remote SSH 连接虚拟机。步骤 4：打开远程目录 ~/bigdata-capstone。步骤 5：新建远程终端，确认不是 Windows PowerShell。

```
whoami
hostname
pwd
java -version

echo "KAFKA_HOME=$KAFKA_HOME"
command -v kafka-topics.sh || true
command -v kafka-server-start.sh || true

# 如果命令不在 PATH, 再找常见安装位置
find /opt /usr/local "$HOME" -maxdepth 4 -name kafka-topics.sh 2>/dev/null | head -n 20
```

如果找到类似 `/opt/kafka/bin/kafka-topics.sh`, 则设置:

```
export KAFKA_HOME=/opt/kafka
export PATH="$KAFKA_HOME/bin:$PATH"

kafka-topics.sh --version
```

版本提醒

不要在本综合项目里随意升级 Kafka。当前 Apache Kafka 最新 Quickstart 已要求 Java 17+; 你的课程 VM 仍以 JDK 1.8 兼容环境为主, 因此本讲沿用 VM 里已经安装并验证可用的 Kafka 版本。

5. 先判断 Kafka 是否已经运行, 不要重复启动

```
jps
ss -lnt | grep 9092 || true

# 如果能成功列 Topic, 说明 Broker 已经可以响应
kafka-topics.sh --bootstrap-server localhost:9092 --list
```

判断标准

最关键的证据不是 `jps` 里有没有一个叫 Kafka 的名字, 而是 9092 是否监听, 以及 Kafka CLI 能否通过 `--bootstrap-server localhost:9092` 正常请求 Broker。

如果 Broker 没有运行, 本地教学 VM 常见两类模式: KRaft 或旧版 ZooKeeper。先识别配置, 不要直接复制互联网上任意启动命令。

```
# 观察配置结构
ls -lah "$KAFKA_HOME/config"

# 旧版 KRaft 常见配置
ls "$KAFKA_HOME/config/kraft/server.properties" 2>/dev/null || true

# 判断 server.properties 中是否使用 ZooKeeper
grep -E '^(process.roles|zookeeper.connect)=' "$KAFKA_HOME/config/server.properties" 2>/dev/null || true
```

课程 VM 如果已经配置好 Kafka, 请沿用原先启动方式。不要在这一步执行 `kafka-storage.sh format`: `format` 是初始化动作, 不是日常启动动作, 误用可能破坏已有 Kafka 数据。

6. 创建项目 Topic : orders_raw

本讲使用 3 个 Partition, Replication Factor=1。单机 VM 只有一个 Broker, 副本因子只能设为 1; 真实集群通常会配置更高副本。

```
kafka-topics.sh --bootstrap-server localhost:9092 --create --if-not-exists --topic orders_raw --partitions 3
--replication-factor 1

kafka-topics.sh --bootstrap-server localhost:9092 --describe --topic orders_raw
```

验收证据

输出中必须能看到 Topic: orders_raw、PartitionCount: 3、ReplicationFactor: 1，并列出 Partition 0/1/2。

7. 第一次不要批量发送：先手工发 1 条、读 1 条

打开 VS Code 的第二个远程终端，启动 Consumer。先不使用 Consumer Group，只是验证 Topic 能正常读。

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw --from-beginning --property print.key=true --property print.partition=true --property print.offset=true --property key.separator='|'
```

再打开第三个远程终端，启动带 Key 的 Producer：

```
kafka-console-producer.sh --bootstrap-server localhost:9092 --topic orders_raw --property parse.key=true --property key.separator='|'
```

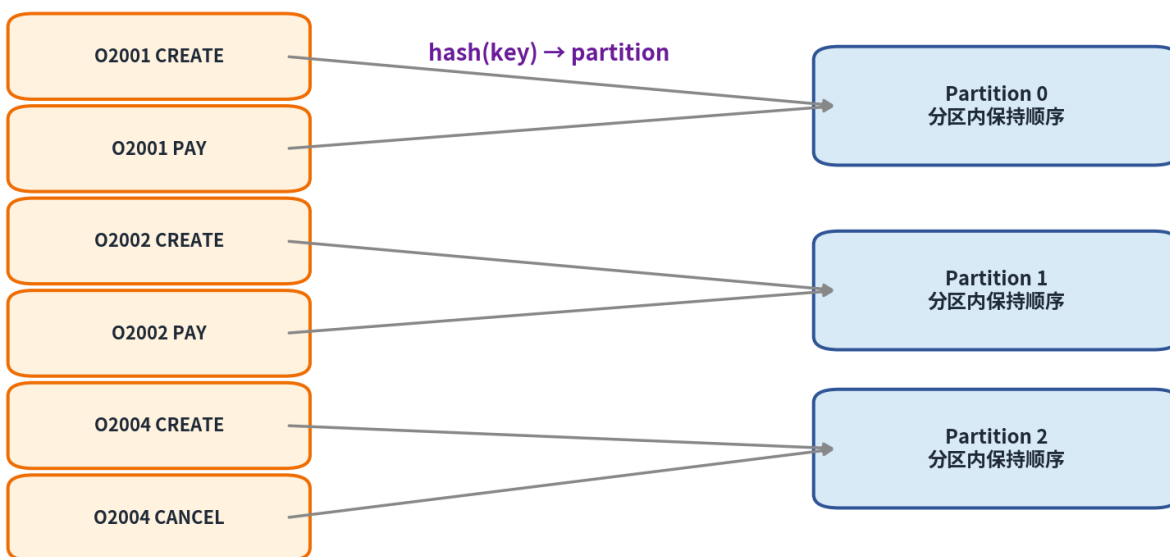
看到 > 提示符后，手工输入一整行（竖线前面是 Key，后面是 JSON Value）：

```
02001|
{"event_id":"TEST001","order_id":"02001","user_id":"U001","city":"Beijing","amount":258.0,"event_type":"CREATE","event_time":"2026-09-29 10:00:05"}
```

回到 Consumer 终端，应该立刻看到 TEST001。此时你已经证明 Producer → Broker → Consumer 最短闭环成立。然后 Ctrl+C 停掉这个测试 Consumer/Producer。

8. 为什么 Message Key 要用 order_id？

为什么本项目用 order_id 作为 Kafka Message Key？



同一个 order_id 的 CREATE/PAY/CANCEL 应使用同一个 key，才能在分区内保持该订单事件顺序。

一个订单通常会依次产生 CREATE、PAY、CANCEL 等事件。我们希望同一个订单的事件进入同一个 Partition，这样至少可以利用 Kafka 的“分区内有序”保证该订单事件的写入顺序。

Kafka 只保证 Partition 内有序

3 个 Partition 之间没有全局顺序。Consumer 输出中看到 E004 跑到 E003 前面，不一定是错误；跨 Partition 的消息不应该用“全局行号”思维理解。

不要中途改变 Partition 数量

基于 Key 的分区是根据 Key 和 Partition 数计算的。实验进行中改变 Partition 数量，某个 Key 后续可能映射到不同 Partition，会破坏“同订单持续落在同一 Partition”的直觉。

9. 把 8 条 JSONL 批量发送到 Kafka

首先确认第 1 讲数据文件仍在：

```
cd ~/bigdata-capstone
wc -l data/order_events.jsonl
head -n 2 data/order_events.jsonl
```

应该是 8 行。为了让 console producer 使用 order_id 作为 Key，需要把每一行临时转换成 order_id|json。本讲代码包已经提供 06_prepare_keyed_events.py。手工操作如下：

```
mkdir -p ~/bigdata-capstone/kafka/lecture6
cd ~/bigdata-capstone/kafka/lecture6

# 把代码包 scripts/06_prepare_keyed_events.py 放到 scripts/ 后
python3 scripts/06_prepare_keyed_events.py ~/bigdata-capstone/data/order_events.jsonl | head -n 2
```

确认输出形如 O2001|{...} 后，再真正发送：

```
python3 scripts/06_prepare_keyed_events.py ~/bigdata-capstone/data/order_events.jsonl | kafka-console-producer.sh
--bootstrap-server localhost:9092 --topic orders_raw --property parse.key=true --property
key.separator='|'
```

注意 TEST001

如果你前面已经手工发送过 TEST001，那么 Topic 中此刻不止 8 条。正式计数实验前，建议重新创建一个干净 Topic，或者创建 orders_raw_l6_clean。PDF 后面给出两种做法。

10. 为了精确验收“8 条”，怎样得到干净实验 Topic？

最安全的方法不是冒险删除已有主 Topic，而是创建一个新的临时 Topic，例如 orders_raw_l6。后面的命令把 TOPIC 名替换成它即可。

```
kafka-topics.sh --bootstrap-server localhost:9092 --create --if-not-exists --topic orders_raw_l6 --partitions 3 --
replication-factor 1

# 再发送 8 条基础事件
python3 scripts/06_prepare_keyed_events.py ~/bigdata-capstone/data/order_events.jsonl | kafka-console-producer.sh
--bootstrap-server localhost:9092 --topic orders_raw_l6 --property parse.key=true --property key.separator='|'
```

如果你确认 orders_raw 只是本地临时实验数据，也可以删除后重建；但删除 Topic 属于破坏性操作，所以本课程默认优先“新建临时 Topic”，不把删除操作写成必做步骤。

11. 从头读取 8 条，并观察 Key / Partition / Offset

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw_l6 --from-beginning --max-
messages 8 --property print.key=true --property print.partition=true --property print.offset=true --property
key.separator='|'
```

你应该能看到 8 条基础事件。不同 Kafka 版本的 console formatter 输出格式可能略有差异，但至少应能观察到 Key 和 Value；如果某个旧版本不支持 print.partition/print.offset，先删掉这两个 property 完成业务验证，再用 Consumer Group 工具观察 Partition 进度。

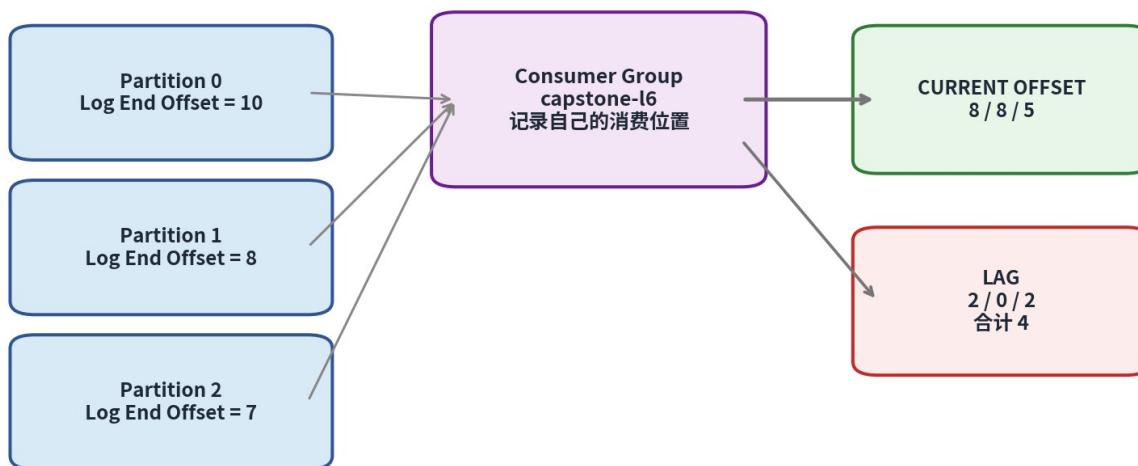
此处真正验收什么？

不是要求 E001~E008 按文件行号严格输出，而是要求 8 个 event_id 都存在；相同 order_id 使用相同 Key；同一 Partition 的 Offset 单调递增。

12. Consumer Group：为什么它不是普通 Consumer 的别名？

Consumer Group 的关键是“保存一套消费进度”。同一个 Group 里的多个消费者会协作分担 Partition；不同 Group 则可以独立从同一个 Topic 读取，不互相抢进度。

Consumer Group 与 Lag：消费者“追上 Kafka 了吗？”



近似理解： $LAG = LOG-END-OFFSET - CURRENT-OFFSET$ （逐 Partition 观察，再看总积压）。

第一次使用 Group capstone-l6 消费 8 条：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw_l6 --group capstone-l6 --from-beginning --max-messages 8 --consumer-property enable.auto.commit=true --consumer-property auto.commit.interval.ms=1000
```

等待 1~2 秒后查询消费组：

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group capstone-l6
```

重点看 TOPIC、PARTITION、CURRENT-OFFSET、LOG-END-OFFSET、LAG。基础 8 条已经被 Group 消费后，总 LAG 应趋近 0。

13. 故意制造 Lag：让消费者停下，再让 Producer 继续写

这是本讲最重要的实验之一。Consumer Group 已经停了，但 Kafka 仍然可以继续接收新事件。代码包提供 3 条 extra events：E009~E011。

```
# 先在代码包目录确认 3 行
wc -l data/order_events_extra.jsonl

# Consumer Group 此刻不运行，发送 3 条新增事件
python3 scripts/06_prepare_keyed_events.py data/order_events_extra.jsonl | kafka-console-producer.sh --
bootstrap-server localhost:9092 --topic orders_raw_l6 --property parse.key=true --property key.separator='|'

# 再查 Group
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group capstone-l6
```

预期现象

LOG-END-OFFSET 前进，但 CURRENT-OFFSET 还停在原位置，因此总 LAG 大约增加 3。由于 3 条消息可能落在不同 Partition，Lag 会分散显示在各行。

现在让同一个 Group 再消费 3 条：

```
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw_l6 --group capstone-l6 --max-
messages 3 --consumer-property enable.auto.commit=true --consumer-property auto.commit.interval.ms=1000

sleep 2
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group capstone-l6
```

LAG 应再次趋近 0。这个实验建立了以后看 Flink Kafka Source Lag 的基础。

14. 一个 Consumer Group 里开两个 Consumer，会发生什么？

orders_raw_l6 有 3 个 Partition。打开两个 VS Code 终端，用完全相同的 group id 启动两个持续 Consumer：

```
# 终端 A
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw_l6 --group capstone-team

# 终端 B
kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders_raw_l6 --group capstone-team
```

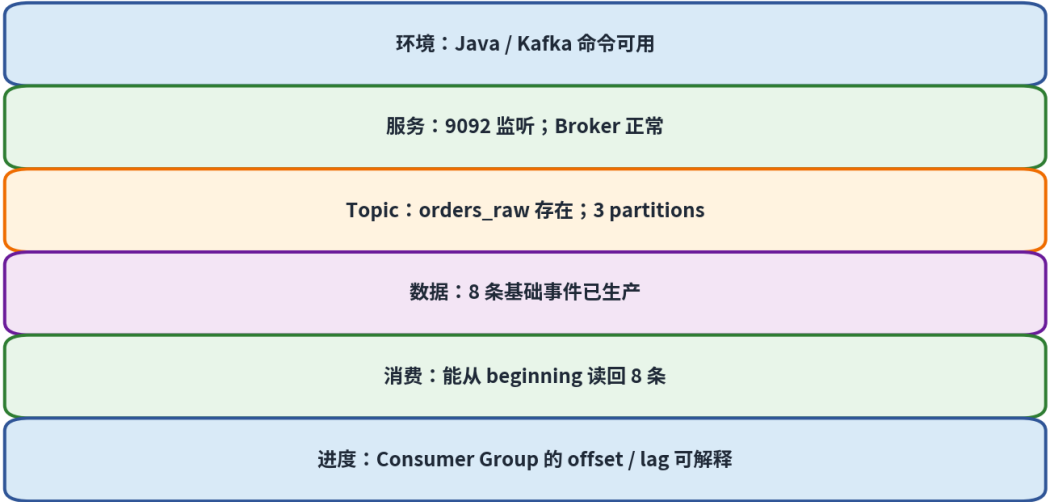
再发送一些事件，你会看到两个消费者分担 Partition。若开 4 个消费者而 Topic 只有 3 个 Partition，至少有一个消费者在某一时刻没有 Partition 可分。这就是“Kafka Source 并行度受 Partition 数约束”的来源之一。

Partition 数 ≠ 越多越好

更多 Partition 可以提高并行能力，但也增加元数据、文件、网络和协调开销。当前小型教学 Topic 用 3 个，是为了观察机制，不是生产系统通用答案。

15. 出错时不要乱试：固定按 6 层排查

Kafka 实验六层证据链：不要只看“命令没报错”



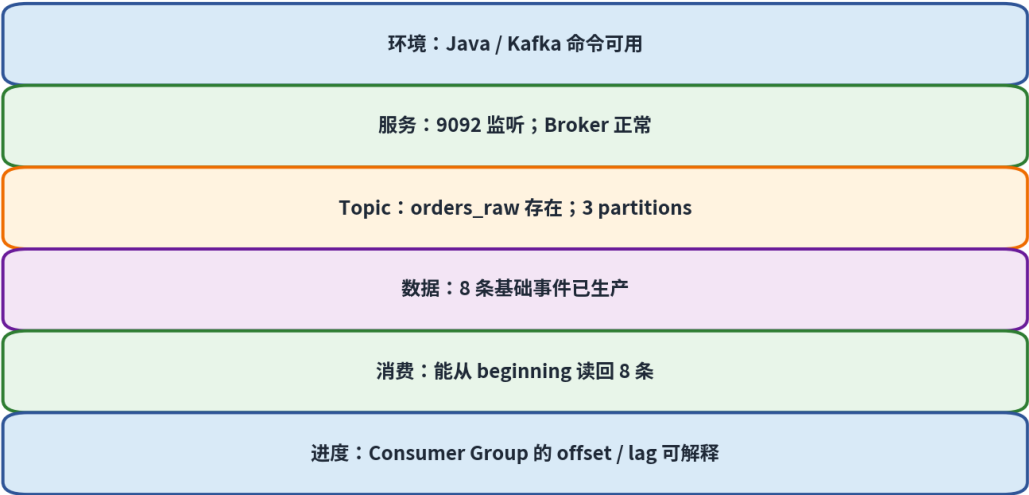
现象	第一检查项	典型原因
Connection to node -1 failed / refused	ss -lnt grep 9092	Broker 没启动或监听地址不对
kafka-*.sh: command not found	echo \$KAFKA_HOME / command -v	Kafka bin 未进 PATH
Topic does not exist	kafka-topics.sh --list/--describe	Topic 名写错或没创建
Consumer 没输出	Topic 是否有消息；--from-beginning	从最新位置等新消息
同订单事件看起来分散	是否真的传了 key；key.separator 是否一致	Producer 没有解析 key
Lag 一直增长	Consumer 是否在跑；下游是否处理太慢	消费能力小于生产速度

不要把“没输出”直接理解成失败

默认 Consumer 可能从“当前最新位置”开始等待新事件；如果历史消息已经存在，需要 --from-beginning，或者使用一个全新的 Consumer Group。

16. 本讲最终验收：必须留下哪些证据？

Kafka 实验六层证据链：不要只看“命令没报错”



证据	你应该能证明
环境证据	kafka-topics.sh --version 能运行；Java 与 Kafka 当前环境兼容
服务证据	9092 监听；CLI 能连接 Broker
Topic 证据	orders_raw_l6 存在；PartitionCount=3
数据证据	8 个基础 event_id 都可从 beginning 读回
Key 证据	同一 order_id 的消息使用同一 Key
进度证据	能解释 CURRENT-OFFSET / LOG-END-OFFSET / LAG
积压证据	Consumer 停止后新增 3 条，Lag 上升；恢复消费后下降
并行证据	同 Group 多 Consumer 能分担 Partition

17. 10 个初学者常见误区

误区	正确理解
Kafka 就是消息队列，消费后消息就没了	Kafka 以日志方式持久保存；是否保留由 retention 等策略决定
Topic 里的消息有全局 Offset	Offset 属于具体 Partition
3 个 Partition 就一定有 3 台机器	Partition 是逻辑分片；本讲 3 个 Partition 都在单 Broker
Consumer Group 就是 Consumer 的名字	Group 代表一套共享消费进度和分区分配关系
同一个 Group 的每个 Consumer 都读全量	同 Group 通常分担 Partition，不重复处理同一 Partition 的同一位置
不同 Group 会互相抢消息	不同 Group 有独立 Offset，可以各自读完整 Topic
Kafka 保证整个 Topic 全局有序	只保证单 Partition 内顺序
Key 只是业务备注	Key 会影响分区；同 Key 通常映射到同一 Partition
Lag=0 说明系统一定健康	只说明消费者进度追上；不代表业务处理结果正确
Broker 启动了就不用验证数据	必须从 Producer、Topic、Consumer、Group Lag 多层验收

18. 练习题：先独立回答，再看答案

- 为什么本项目把 order_id 作为 Message Key，而不是 event_id？
- 一个 Topic 有 3 个 Partition，同一个 Consumer Group 开 5 个 Consumer，能否 5 个都持续有数据可读？为什么？
- CURRENT-OFFSET=12、LOG-END-OFFSET=17，Lag 是多少？它说明什么？
- 为什么 E004 可能比 E003 先被 Consumer 打印出来，却不一定说明 Kafka 乱序？

5. 如果同一个 Topic 被 Flink 和一个调试脚本同时消费，为什么通常应该使用不同 Consumer Group?
6. 为什么本讲不要求启动 HDFS/YARN?

19. 参考答案

1. order_id 把同一个订单的 CREATE/PAY/CANCEL 归在同一 Key 下，从而尽量进入同一 Partition，利用分区内有序；event_id 每条都不同，达不到这个目的。
2. 不能保证。一个 Partition 在同一 Group 内同一时刻只会分配给一个 Consumer；只有 3 个 Partition 时，最多 3 个 Consumer 同时拥有 Partition。
3. Lag=5，表示该 Group 在这个 Partition 上还有约 5 条消息未追上。
4. E003/E004 可能属于不同 order_id、不同 Partition。Kafka 不保证跨 Partition 全局顺序。
5. 不同 Group 有独立消费进度；若共用 Group，它们会协作分担消息，调试脚本可能“抢走”Flink 应处理的 Partition。
6. 本讲只验证 Kafka 实时事件链。启动无关服务会增加变量，违背“先把单层系统跑通”的排错原则。

20. 版本提醒与本讲收束

当前官方 Kafka 与课程 VM 的差异

截至 2026-09-29，Apache Kafka 官方 Quickstart 的最新版本为 4.3.1，要求 Java 17+。本课程 VM 仍以 JDK 1.8 为 Hadoop 3.3.5 的统一环境，因此综合项目不在此处强制升级 Kafka；优先使用 VM 中已经安装、与现有 Java 兼容的版本。后续需要新 Kafka/Flink 时，再采用多 JDK 并存。

到这里你应该能独立解释：Producer 产生事件，Broker 把事件追加到 Topic 的 Partition；Offset 标记分区内位置；Message Key 决定同订单事件的分区；Consumer Group 维护自己的消费位置；Lag 表示消费者离最新日志还有多远。

下一讲预告

第 7 讲会让 Flink 真正接入 Kafka：Kafka Source → JSON 解析 → Event Time → Watermark → 1 分钟窗口 → 城市实时销售额。到那时你会直接用上本讲建立的 Topic、Key、Partition、Offset、Consumer Group 和 Lag 心智模型。

参考资料（官方）

Apache Kafka Quickstart: <https://kafka.apache.org/quickstart/> （用于核对 Topic、Producer、Consumer 基本 CLI 与当前版本 Java 要求）